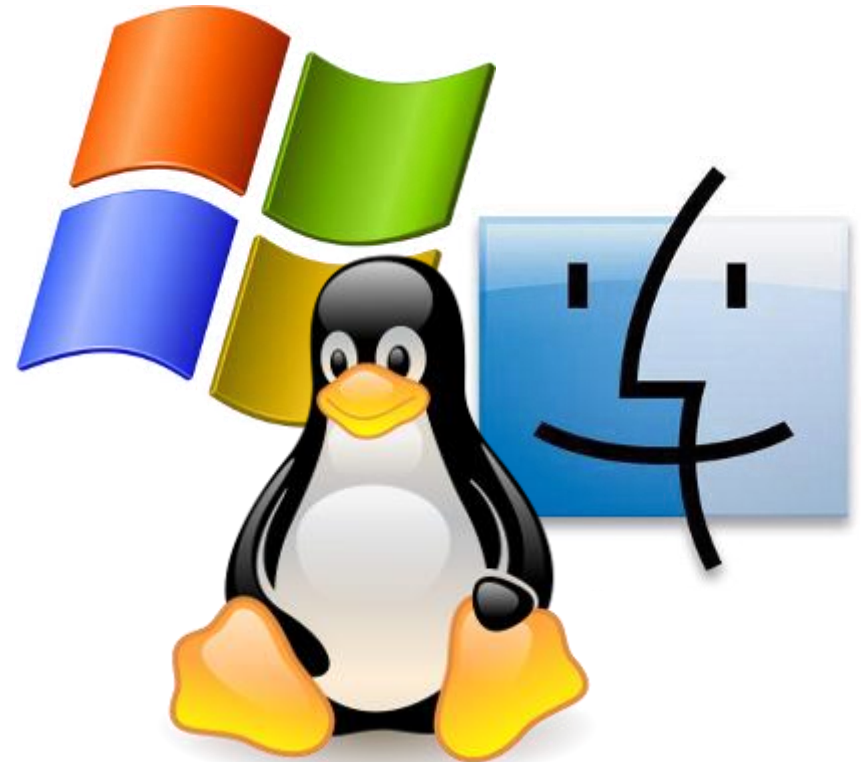


1 – Einführung

Tobias Werner M. Sc.
Uni Bayreuth – Angewandte Informatik 3
Leibniz-FH Hannover

`tobias.werner.doz@leibniz-fh.de`

(in Teilen nach der Vorlesung
"Betriebssysteme" von Prof. Dominik Henrich
an der Universität Bayreuth)



[Logos von Windows 7, Linux und Mac OS X]

Vortragsübersicht

1 – Einführung

- Organisatorisches
- Inhalt der Vorlesung

- Betriebssysteme als Begriff
- Historische Entwicklung
- Systemaufrufe
- Betriebssystemstruktur

Organisation (1)

Zielgruppe

- Studierende Technische Informatik im 3. Semester

Voraussetzungen

- etwas Verständnis von C und C++
- Basisverständnis Englisch

Hard- und Software

- halbwegs aktueller PC
- installierter Compiler + IDE
 - Windows: Visual Studio Community
 - Linux, macOS: Qt Creator + GCC

Organisation (2)

Ablauf

pro Kapitel drei Teile:

- eine Vorlesung (ca. 1 Stunde)
- eine Übung (ca. 1 Stunde)
- eine Übungskorrektur (ca. 1 Stunde)

Vorlesung

Folien:

- tagesaktuell im e-Learning-Portal der DHSN
- bei Bedarf Notizen machen, es gibt kein ausformuliertes Skript!

interaktives Lernen:

- Zwischenfragen sind erlaubt und erwünscht
- hin und wieder Mentimeter-Umfragen
- **nach zusätzlicher Pause rufen, wenn Bedarf besteht!**

Organisation (3)

Übung

Aufgaben:

- tagesaktuell im e-Learning-Portal
- in Umfang und Schwierigkeit ähnlich der Klausur

Bearbeitung:

- in festen Gruppen zwischen 2 und 4 Teilnehmern
- Lösung in der Gruppe erstellen
- **bei Bedarf den Dozenten um Hilfe bitten!**

Korrektur:

- auf Wunsch Vorrechnen der eigenen Lösung im Plenum
 - Korrektur durch das Plenum und den Dozenten
 - sonst Vorrechnen durch den Dozenten
- Musterlösungen jeweils kurz nach den Übungsterminen

zudem: Probeklausur mit Musterlösung kurz vor Kursende

Organisation (4)

Klausur

Termin: dem Dozenten nicht bekannt

Ablauf: schriftlich, 1 h Schreibdauer

Hilfsmittel: vermutlich keine

nach Abstimmung im Kurs möglich:

→ ein DIN-A4-Zettel handschriftlich mit Notizen

→ dann aber keinerlei Auswendiglernfragen!

Benotung:

- ca. 25 Punkte aus ca. 3 Aufgaben
- Zweit- und Drittversuche werden nicht leichter

Inhalt:

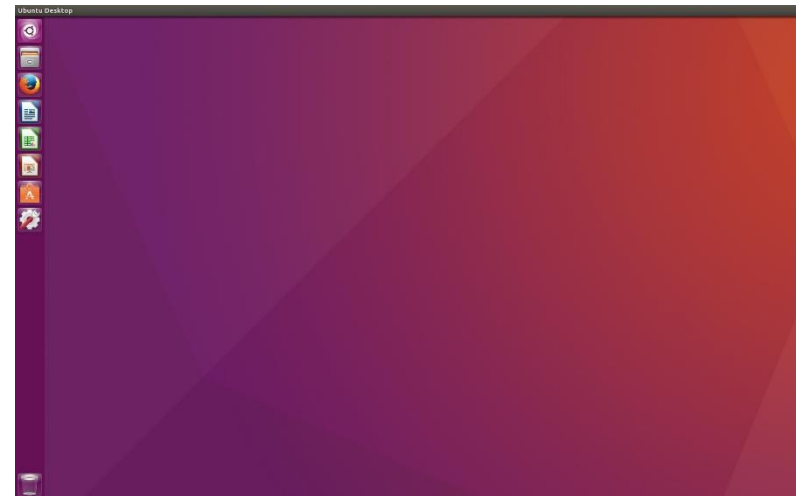
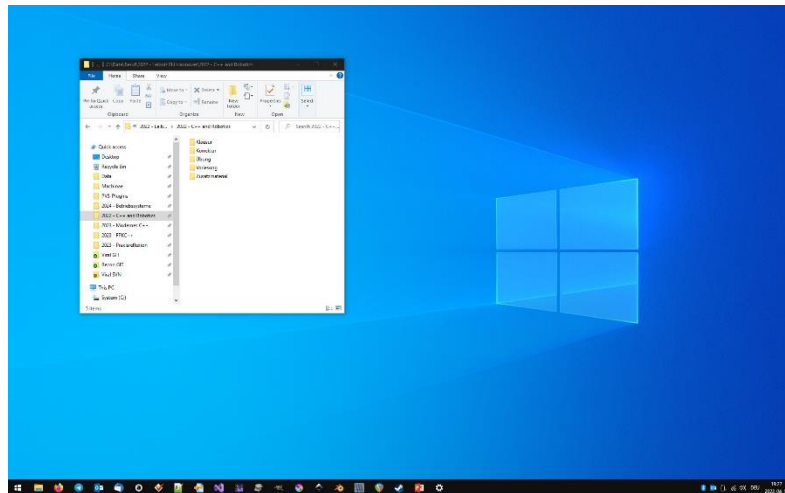
- der gesamte Umfang der Veranstaltung **ohne grobe Themeneinschränkung**
 - Lernen auf Lücke ist **nicht** möglich
 - Stoffplan Basiswissen / Kürwissen / unwichtig kurz vor der Klausur
- ca. 25% Inhaltsabfrage (z. B. Definitionen korrekt wiedergeben)
- ca. 50% Anwendung (z. B. Algorithmen von Hand nachvollziehen)
- ca. 25% Theorie (z. B. leichte Beweise)

Inhalt der Vorlesung (1)

Motivation

Betriebssysteme (OS, operating systems) sind...

- ... die Basis von jedem modernen Rechensystem
- ... eine Grundlage der Informatik
- ... von jedem Informatiker zu kennen
- ... die Voraussetzung zur Verwaltung von PC-Pools
- ... in vielen praktischen Anwendungsfällen einsetzbar



[Desktops von Windows 11 und Ubuntu]

Inhalt der Vorlesung (2)

Lernziele

theoretische Grundlagen von Betriebssystemen:

- grundsätzlicher Aufbau
- verwendete Verfahren (z. B. Algorithmen, Datenstrukturen)
- Auswahlkriterien für praktische Anwendungsfälle

Abgrenzung:

- keine Einschränkung auf ein bestimmtes Betriebssystem
- keine Implementierungsdetails (z. B. innere Umsetzung der Win32-API)
- keine Administrationsarbeiten (z. B. Installation, Konfiguration)

allgemein:

- Methoden zur effizienten Verwaltungen von zeitlichen und räumlichen Ressourcen
→ können auch außerhalb von Betriebssystemen eingesetzt werden

Inhalt der Vorlesung (3)

Gliederung

1 – Einleitung

→ Wie unterscheiden sich Betriebssysteme?

2 – Prozessverwaltung (2 Teile)

→ Wie können mehrere Vorgänge gleichzeitig (fehlerfrei) bearbeitet werden?

3 – Speicherverwaltung (2 Teile)

→ Wie kann auf sehr großen Speicher effizient zugegriffen werden?

4 – Dateiverwaltung

→ Wie können große, dauerhafte Datenmengen effizient verarbeitet werden?

5 – Ein- und Ausgabeverwaltung

→ Wie können externe Geräte effizient an die CPU angebunden werden?

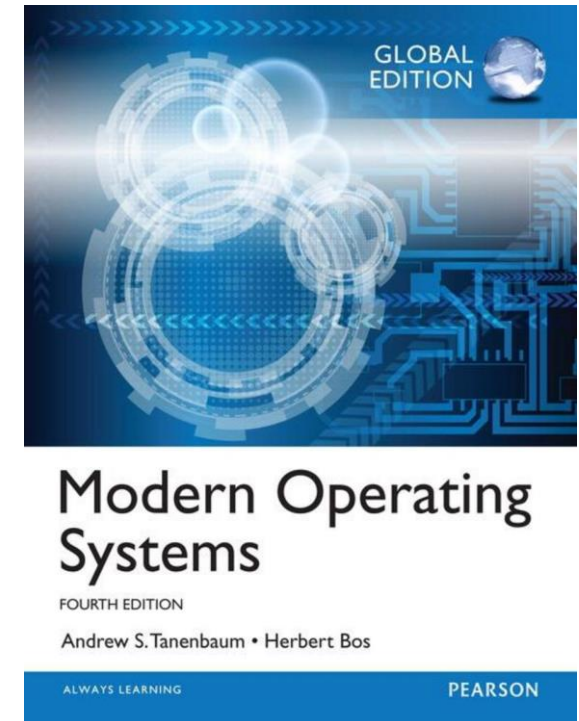
6 – Systemsicherheit

→ Wie kann das Betriebssystem vor Bedrohungen schützen?

Inhalt der Vorlesung (4)

Lehrbücher

- [Tanenbaum15]
Tanenbaum, Herbert: "Modern Operating Systems"
4th Edition, Pearson Education, 2015
- [Tanenbaum09]
Tanenbaum: "Moderne Betriebssysteme"
2., überarbeitete Auflage, Pearson Studium, 2009
- [Brause01]
Brause: "Betriebssysteme – Grundlagen und Konzepte"
2., überarbeitete Auflage, Springer, 2001
- [Beck99]
Beck et al.: "Linux Kernelprogrammierung – Algorithmen und Strukturen der Version 2.2"
5. Auflage, Addison-Wesley, 1999
ISBN 978-3827314765



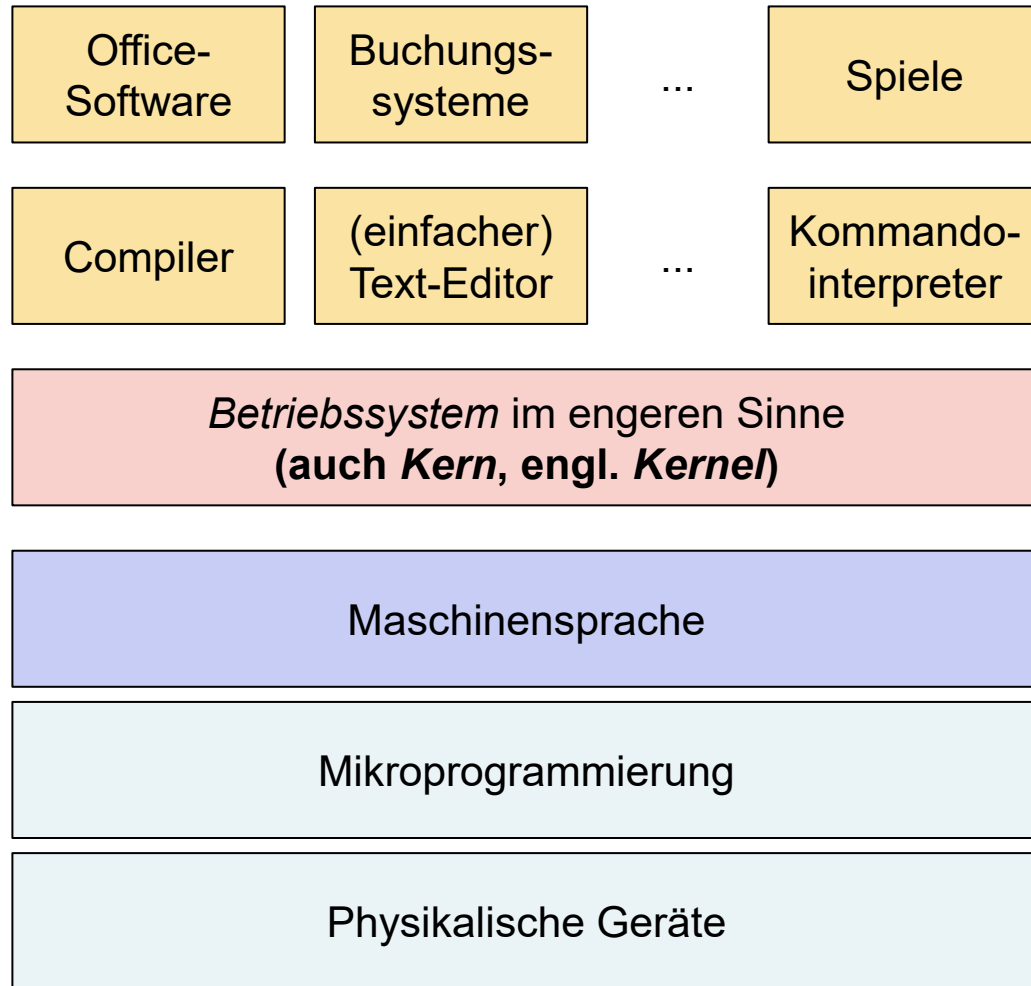
Vortragsübersicht

1 – Einführung

- Organisatorisches
- Inhalt der Vorlesung
- Betriebssysteme als Begriff
- Historische Entwicklung
- Systemaufrufe
- Betriebssystemstruktur

Betriebssysteme als Begriff (1)

Aufbau von Rechensystemen



Anwendungssoftware

- löst Probleme der (End-)Nutzer
- Anwendungssoftware läuft im *Benutzermodus (user mode)* mit eingeschränktem Zugang (z. B. auf Speicher und Hardware)

Systemsoftware

- dient der Verwaltung und des Betriebs des Rechners selbst
- Betriebssystem läuft im privilegierten *Kernmodus (kernel mode)* mit vollständigem Zugang zur gesamten Hardware und zu allen Befehlen

Hardware

- führt Software aus

[nach Tanenbaum15]

Betriebssysteme als Begriff (2)

Definitionsansätze

Bottom-Up-Definition:

Betriebssystem als *erweiterte Maschine*

- OS versteckt reale Hardware hinter vereinfachter, abstrakter Schicht
- Umsetzung durch sogenannte *Systembefehle* bzw. *Systemaufrufe*

- neue Sicht auf Hardware heißt *virtuelle Maschine*
- häufig mehrere, gestapelte virtuelle Maschinen in einem OS

Beispiel:

die Systemaufrufe `read()` und `write()` für den Dateizugriff "verstecken" die Ansteuerung einer SSD

Top-Down-Definition:

Betriebssystem zur *Ressourcenverwaltung*

- OS teilt beschränkte Ressourcen (z. B. Rechenzeit, Speicher) konkurrierenden Aufgaben zu (z. B. Programmen, Nutzern)
- Zuteilungsverfahren heißt *Allokation* (Verb: *allozieren*, nicht *allokieren*!)
- zwei Dimensionen für die Allokation zu berücksichtigen: Zeit und Raum

Beispiel:

zwei Programme speichern gleichzeitig zwei Dateien, OS legt fest wann (Zeit) und wo (Raum) auf SSD geschrieben wird

[nach Tanenbaum15]

Betriebssysteme als Begriff (3)

Betriebsarten für Betriebssysteme

Stapelbetrieb (batch mode)

- Aufträge / Jobs / Programme werden zu Paketen geschnürt und nacheinander abgearbeitet

Dialogbetrieb (interactive mode)

- Benutzer bekommt interaktiv auf jede Aktion eine direkte Reaktion angezeigt

Echtzeitbetrieb (realtime mode)

- Programme werden innerhalb einer vorgegebenen Zeit abgeschlossen

Mehrprogrammbetrieb (multitasking mode)

- mehrere Programme laufen gleichzeitig auf einem Rechner

Mehrbenutzerbetrieb (multiuser mode)

- mehrere Benutzer arbeiten gleichzeitig an einem Rechner

[nach Tanenbaum15]

Betriebssysteme als Begriff (4)

Betriebsarten heutiger Betriebssysteme

moderne Betriebssysteme laufen üblicherweise gleichzeitig im...

- Dialogbetrieb
 - Gewährleistung von akzeptablen Reaktionszeiten nötig
- Mehrprogrammbetrieb (pro Nutzer)
- Mehrbenutzerbetrieb
 - OS muss begrenzte Ressourcen (z. B. Speicher) auf Nutzer und Programme verteilen

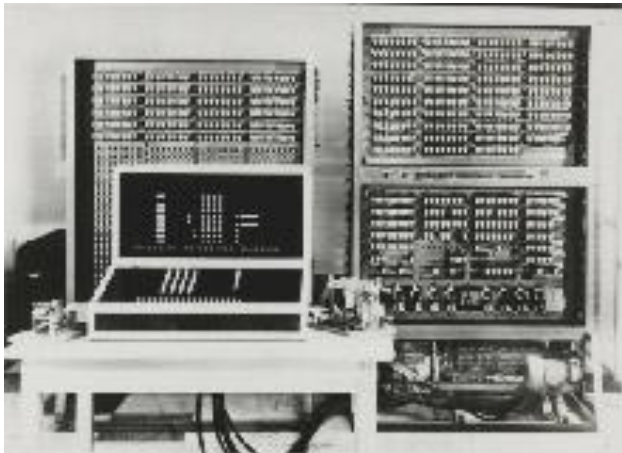
moderne Betriebssysteme können üblicherweise...

- keinen Echtzeitbetrieb
 - feste Reaktionszeit ist nicht gewährleistet (z. B. bei Updates, voller Systemlast)
- keinen Stapelbetrieb
 - entsprechende Funktion (z. B. für Programme) ist nicht ins OS direkt integriert...
 - ... wird aber eventuell von System- oder Anwendungssoftware unterstützt

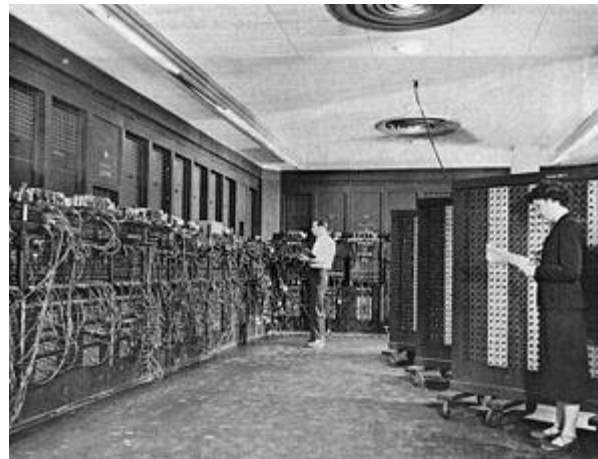
Historische Entwicklung (1)

1. Generation

- zeitliche Einordnung: ca. 1945 bis 1955
- Rechnertechnologie: Relais und Röhren
- Betriebssystem-Merkmale: kein OS!
- Programmierung über Steckbretter oder Maschinensprache



Z3, 1941 (Nachbau)



ENIAC, 1946

[nach Tanenbaum15]

Historische Entwicklung (2)

2. Generation

- zeitliche Einordnung: ca. 1955 bis 1965
- Rechnertechnologie: einzelne Transistoren in *Großrechnern (mainframes)*
- Betriebssystem-Merkmale: *Stapelverarbeitung (batch processing)* von Aufträgen (*jobs*)
- Programmeingabe über Lochkarten, Ausgabe über Ausdrucke
- Später: Zwischenspeicherung und Sammlung der Ein- und Ausgabe auf Magnetbändern

IBM 704, 1954



[nach Tanenbaum15]

Historische Entwicklung (3)

3. Generation

- zeitliche Einordnung: ca. 1965 bis 1980
- Rechnertechnologie: mehrere integrierte Bausteine (*integrated circuit*, IC) in *Minicomputer*
- Betriebssystem-Merkmale:
 - Mehrprogrammbetrieb
 - Spooling
(Simultaneous Peripheral Operation On Line)
 - Dialogsysteme

DEC PDP-7, 1965



[nach Tanenbaum15]

Historische Entwicklung (4)

4. Generation

- zeitliche Einordnung: ca. 1980 bis heute
- Rechnertechnologie: vollständig auf einem Chip integrierte Schaltkreise (*Very Large Scale Integration, VLSI*) in *Mikrocomputer*
- Betriebssystem-Merkmale:
 - Betriebssystem auf Diskette (disk operation system, DOS, z. B. MS-DOS)
 - fenster-basierte Benutzerschnittstelle
 - verteilte Betriebssysteme



IBM PC, 1981



Commodore 64, 1986



Modularer Desktop-PC, 2020

[nach Tanenbaum15]

Historische Entwicklung (5)

5. Generation

- zeitliche Einordnung: ca. 1995 bis heute
- Rechnertechnologie: portable, batterie-betriebene (Multicore-)Rechner in Laptops bzw. (Sub-)Notebooks
- Betriebssystem-Merkmale:
 - fenster-basierte Benutzerschnittstelle
 - Energiemanagement



Notebook, 1992



Notebook, 2022

[nach Tanenbaum15]

Historische Entwicklung (6)

6. Generation

- zeitliche Einordnung: ca. 2005 bis heute
- Rechnertechnologie: Multicore-Rechner nur mit Touch-Screens als PDAs (Persönliche Digitale Assistenten) zu bedienen
- Betriebssystem-Merkmale:
 - Energiemanagement
 - touch-basierte Benutzerschnittstelle
 - Cloud Computing



Tablet



Smartphone

[nach Tanenbaum15]

Vortragsübersicht

1 – Einführung

- Organisatorisches
- Inhalt der Vorlesung
- Betriebssysteme als Begriff
- Historische Entwicklung
- Systemaufrufe
- Betriebssystemstruktur

Systemaufrufe (1)

Was macht ein Betriebssystem für Programmierer?

Aufgabe: Schreibe den Text "hallo welt" auf einen Datenträger

Umsetzung ohne Betriebssystem:

- Schwierig... was kompiliert und startet unsere Programme?
- Was wäre denn grundsätzlich zu tun?
 - hardwarenahe Programmierung (z. B. in Maschinencode)
 - herausfinden, welches Modell Datenträger verbaut ist (z. B. Hersteller, Art)
 - entscheiden, wo auf dem Datenträger die Daten hinkommen
 - den Datenträger individuell ansteuern (z. B. Spannung auf Leiterbahnen setzen)
 - Konflikte und Fehler erkennen (z. B. wenn der Datenträger voll ist)

Umsetzung mit Betriebssystem:

```
int fd = open("output.txt", O_RDWR);    // Öffne Datei mit Dateihandle fd, ...
write(fd, "hallo welt", 10);            // ... schreibe 10 Zeichen hinein, ...
close(fd);                              // ... und gebe Datei wieder frei.
```

- Betriebssystem exponiert Systemaufrufe (hier `read()`) als Schnittstelle

Systemaufrufe (2)

Portable Operation System

- POSIX-1003.1 ist IEEE-Standardisierung von ca. 100 Systemaufrufen
- wird von den meisten UNIX-Varianten unterstützt

Beispiele Prozessverwaltung:

(→ Kapitel 1)

Aufruf	Beschreibung
<code>pid = fork()</code>	Erzeugen eines Kindprozesses identisch zum Vater
<code>waitpid(pid, &statloc, options)</code>	Warten auf Terminierung des Kindes
<code>exit(status)</code>	Prozess beenden und Status zurückliefern

Beispiele Dateiverwaltung:

(→ Kapitel 3)

Aufruf	Beschreibung
<code>fd = open(file, how, ...)</code>	Datei zum Lesen und/oder zum Schreiben öffnen
<code>s = close(fd)</code>	Offene Datei schließen
<code>n = read(fd, buffer, nbytes)</code>	Daten aus Datei in Puffer lesen
<code>n = write(fd, buffer, nbytes)</code>	Daten vom Puffer in Datei schreiben

[nach Tanenbaum15]

Systemaufrufe (3)

Verzeichnis- und Dateiverwaltung:

(→ Kapitel 3)

Aufruf	Beschreibung
<code>s = mkdir(name, mode)</code>	Erzeugen eines neuen Verzeichnisses
<code>s = rmdir(name)</code>	Löschen eines leeren Verzeichnisses
<code>s = link(name1, name2)</code>	Neuer Eintrag name2 zeigt auf name1
<code>s = unlink(name)</code>	Verzeichniseintrag löschen
<code>s = mount(special, name, flag)</code>	Dateisystem einbinden
<code>s = unmount(special)</code>	Eingebundenes Dateisystem entfernen
<code>s = chdir(dirname)</code>	Wechsel des aktuellen Verzeichnisses

Verschiedenes:

(→ Kapitel 3, 5)

Aufruf	Beschreibung
<code>s = chmod(name, mode)</code>	Ändern der Dateirechte
<code>s = kill(pid, signal)</code>	Signal an einen Prozess schicken
<code>seconds = time(&seconds)</code>	Zeit seit 1. Januar 1970 erfragen

[nach Tanenbaum15]

Systemaufrufe (4)

Win32 Application Programming Interface

- Sammlung von Betriebssystemaufrufen zur Kommunikation mit Windows ab 95
- proprietäre Microsoft-Implementierung ähnlich, aber nicht gleich zu POSIX

UNIX	Win32	Beschreibung
fork()	CreateProcess()	Neuen Prozess erzeugen
waitpid()	WaitForSingleObject()	Auf das Ende eines Prozesses warten
exit()	ExitProcess()	Auführung beenden
open()	CreateFile()	Neue Datei erzeugen oder Datei öffnen
close()	CloseHandle()	Datei schließen
read()	ReadFile()	Daten aus einer Datei lesen
write()	WriteFile()	Daten in eine Datei schreiben
mkdir()	CreateDirectory()	Neues Verzeichnis erzeugen
rmdir()	RemoveDirectory()	Leeres Verzeichnis löschen
link()	-	Win32 unterstützt keine harten POSIX-Links
unlink()	DeleteFile()	Existierende Datei löschen
mount()	-	Win32 unterstützt kein POSIX-Einbinden
umount()	-	Win32 unterstützt kein POSIX-Einbinden
chdir()	SetCurrentDirectory()	Arbeitsverzeichnis ändern
chmod()	-	Keine POSIX-Rechte in Windows, stattdessen ACLs
kill()	-	Win32 unterstützt keine POSIX-Signale
time()	GetLocalTime()	Aktuelle Zeit erfragen

[nach Tanenbaum15]

Systemaufrufe (5)

Ablauf eines Systemaufrufs

1. Aufruf durch Anwendung:

- 1a. Anwendung legt Parameter (z. B. zu lesende Datei) auf den *Stapel* (= Zwischenspeicher)
- 1b. Anwendung ruft Bibliotheksfunktion (z. B. `read`) für den Systemaufruf auf

2. Aufruf durch Bibliotheksfunktion:

- 2a. Bibliotheksfunktion legt *Code* (= Identifikationsnummer) für Systemaufruf in CPU-Register
- 2b. Bibliotheksfunktion löst Systemaufruf in den OS-Kern aus (*kernel trap*)

3. Abarbeitung im Betriebssystemkern:

- 3a. *Dispatcher* (= Verteiler) im OS-Kern schlägt Kernelfunktion zu Systemaufruf-Code in Tabelle nach und ruft diese Funktion auf
- 3b. Kernelfunktion läuft durch und springt nach Abschluss zurück zur Bibliotheksfunktion

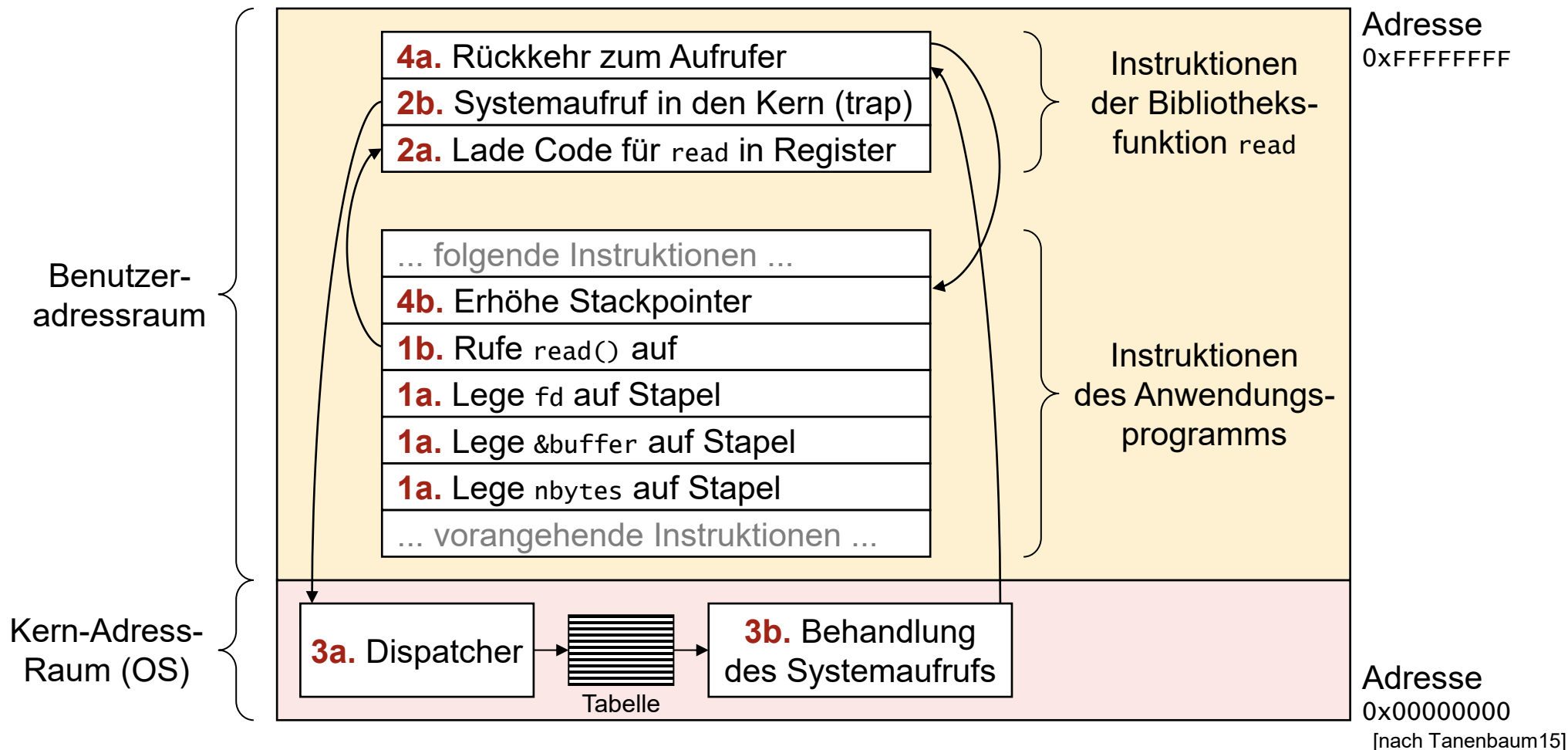
4. Rückkehr und Aufräumen:

- 4a. Bibliotheksfunktion springt zu Anwendung zurück
- 4b. Anwendung bereinigt den Stapel und setzt die Ausführung fort

Systemaufrufe (6)

Beispiel für einen konkreten Systemaufruf

Aufruf von `read(fd, &buffer, nbytes)`



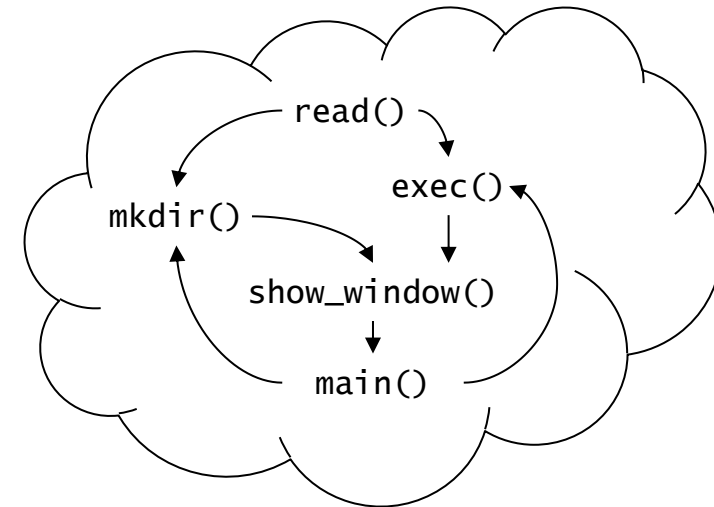
Betriebssystemstruktur (1)

Monolithische Struktur

Grundform:

OS und Anwendungen sind zusammen
ein einziges Programm ohne Struktur

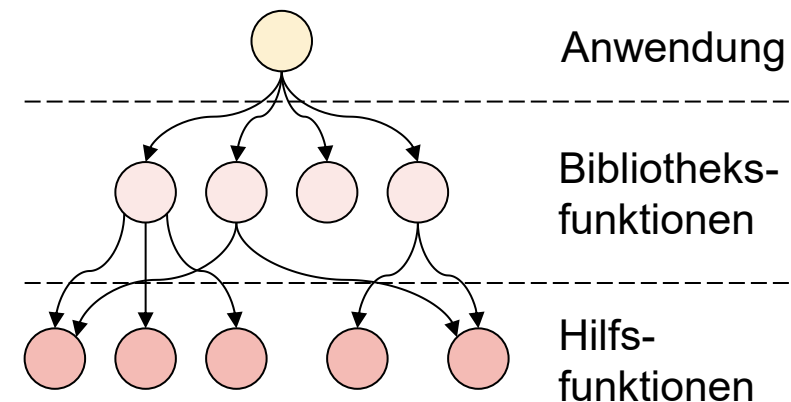
- jede Funktion kann jede andere Funktion aufrufen
- die gesamte Information ist überall sichtbar
- problembehaftet (z. B. Sicherheit, Wartung)



verbesserte Variante:

dreischichtige Struktur zwischen Anwendung,
Systemaufrufen und OS-Hilfsfunktionen

- pro Systemaufruf gibt es eine Bibliotheksfunktion
- OS-interne Hilfsfunktionen stehen Anwendungen nicht zur Verfügung
- etwas sicherer und besser zu warten



[nach Tanenbaum15]

Betriebssystemstruktur (2)

Geschichtete Struktur

verallgemeinerter Ansatz:

Betriebssystem besteht aus einer Hierarchie von Schichten

- eine Schicht baut (z. B. bezüglich genutzten Features) nur auf tieferen Schichten auf
- weitere Verbesserung bezüglich Sicherheit und Wartbarkeit

Beispiel: THE-System der TH Eindhoven, E. W. Dijkstra, 1968

ähnlich: Ring- oder Zwiebelschalen-Architekturen (z. B. MULICS)

Schicht	Funktionalität	Effekt	
6	der Benutzer	–	
5	Benutzerprogramme	(Funktion des Programms)	
4	Ein-/Ausgabeschichten	abstrakte Geräte mit gepufferter E/A	→ Kapitel 4, 5
3	Kommunikationsverwaltung	eine Bedienkonsole pro Prozess	→ Kapitel 4
2	Speicherverwaltung	ein Adressraum pro Prozess	→ Kapitel 3
1	Prozessverwaltung	Mehrprogrammbetrieb	→ Kapitel 2

[nach Tanenbaum15]

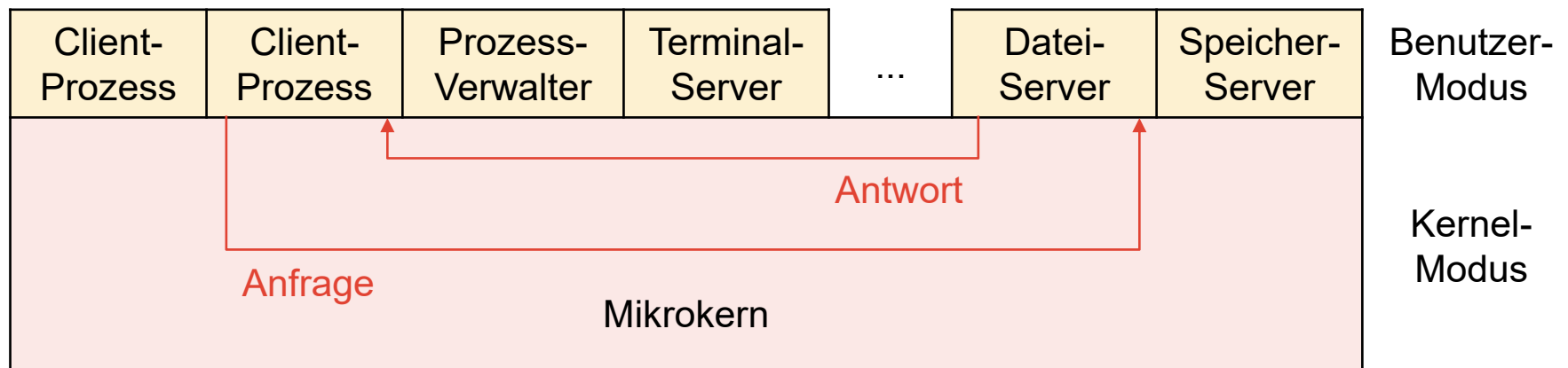
Betriebssystemstruktur (3)

Client/Server-Struktur

Idee:

lagere möglichst viele OS-Funktionen in höhere Schichten aus,
übrig bleibt nur ein minimaler *Mikrokern* (*micro kernel*)

- Mikrokern übermittelt ausschließlich Nachrichten zwischen Client-Prozessen (z. B. Anwendungen) und Server-Prozessen (z. B. Datei-Server des OS)
- kleinere, einfachere OS-Bestandteile (z. B. weniger Abhängigkeiten)
- robustere OS-Bestandteile (z. B. Neustart abgestürzter Prozesse)
- Verwendung z. B. in verteilten Betriebssystemen möglich



[nach Tanenbaum15]

Betriebssystemstruktur (4)

Verteilte Betriebssysteme

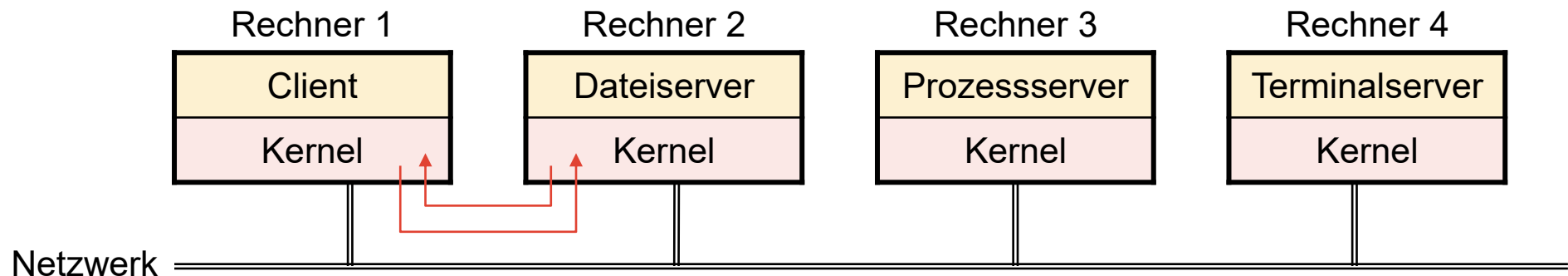
Idee:

verwende viele Rechner (z. B. für komplizierte Berechnungen) mit jeweils eigenem OS, aber lasse die Rechner wie ein einzelner Rechner mit einem einzigen OS aussehen

- Client/Server-Modell mit Nachrichtenaustausch
- Client schickt Auftrag an Server, welcher Antwort zurückschickt

Beispiele:

- Großrechner, Number Cruncher
- Rechensysteme der Firma Cray
- OS "Amoeba" von Tannenbaum bis 1998



[nach Tanenbaum15]

Zusammenfassung

Begrifflichkeiten

- Anwendungssoftware vs. Systemsoftware vs. Betriebssystem
- Betriebssystem als virtuelle Maschine oder als Ressourcenverwalter
- Stapel-, Dialog-, Echtzeit-, Mehrprogramm-, Mehrnutzerbetrieb

Systemaufrufe

- bieten Programmierschnittstelle für Programme an
- POSIX-Standard und Win32-API
- Wechsel vom Benutzermodus in den Kernelmodus

Betriebssystemstrukturen

- monolithisch, geschichtet, Client/Server, verteilt
- Kapselung in kleine, unabhängige Einheiten vorteilhaft